

Scaling applications



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

1

What is a distributed application

Computing on multiple threads/devices/nodes in parallel

What can run in parallel

- 💡 Data loading
- 💡 Training
- 💡 Inference

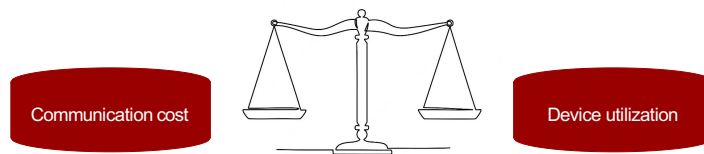


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

The key take away

⚠ Distributed computation is not always beneficial, its a trade-off:



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

Devices

Three common types of devices

- 💡 CPU
 - 🔥 General compute unit
 - 🔥 2-128 parallel operations
- 💡 GPU
 - 🔥 Rendering unit
 - 🔥 1.000-10.000 parallel operations
- 💡 TPU
 - 🔥 Specialized unit
 - 🔥 32.000 - 128.000 parallel operations



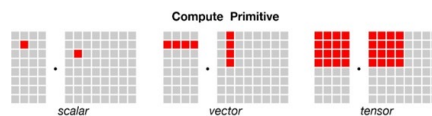
CPU



GPU



TPU



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

Device memory

🔥 Equally important is the amount of memory you have available

With more memory you get

- 💡 Faster data transfer
- 💡 Possibility of higher data modalities
- 💡 Larger models

	CPU	GPU	TPU
Standard	34-64 GiB	12 GiB	64 GiB
Maximum	2 TiB	80 GiB	32 TiB

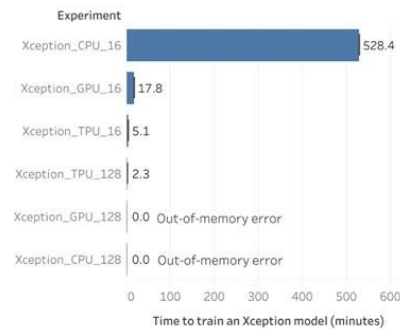
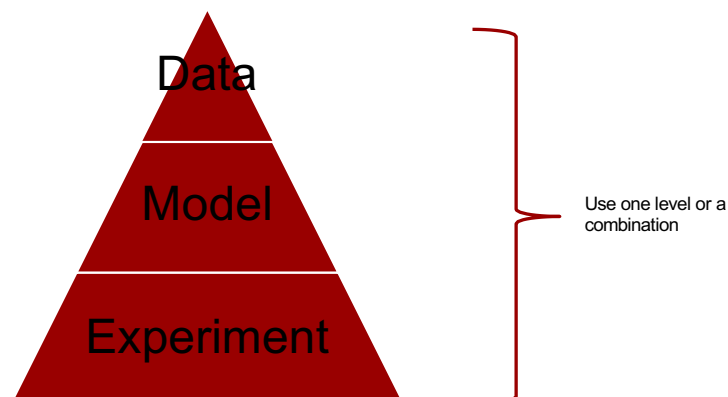


Figure 3: CPUs vs GPUs vs TPUs for training an Xception model for 12 epochs. Y-Axis labels indicate the choice of model, hardware, and batch size for each experiment. Increasing the batch size to 128 for TPUs resulted in an additional ~2x speedup.

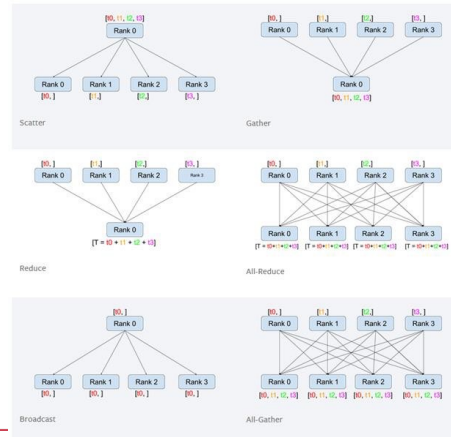
Many layers of distributed computations



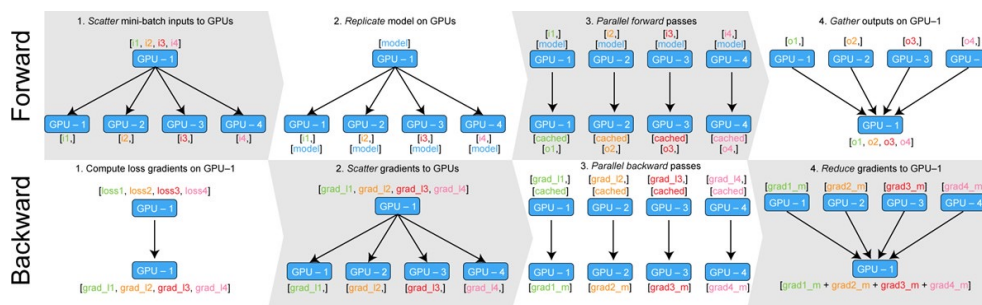
Basic communication operations

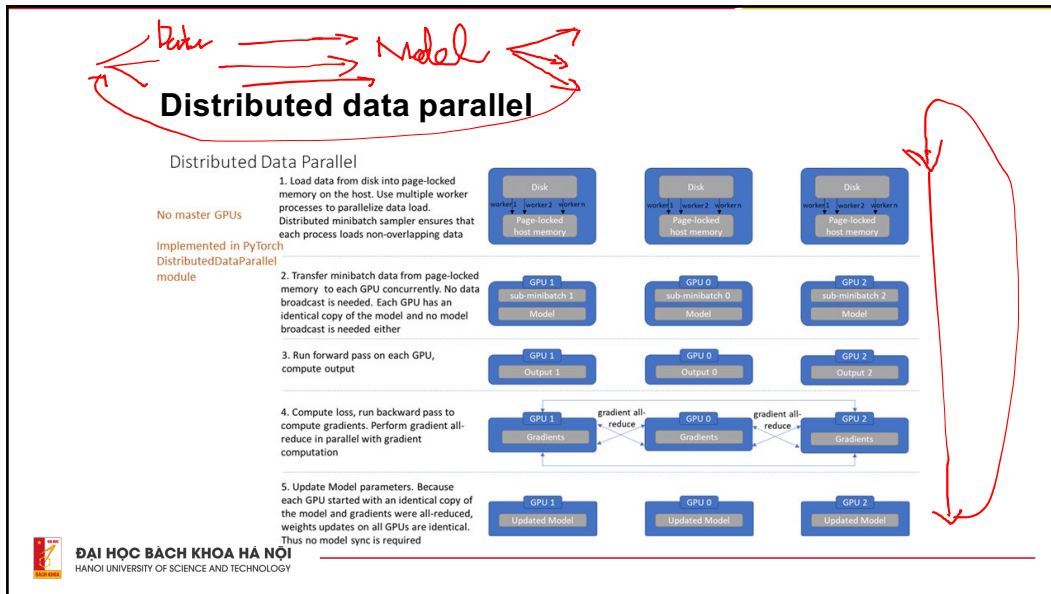
- 💡 Scatter
- 💡 Gather
- 💡 Reduce
- 💡 Broadcast
- 💡 All-gather
- 💡 All-reduce

Rank 0: main
Rank >0: worker

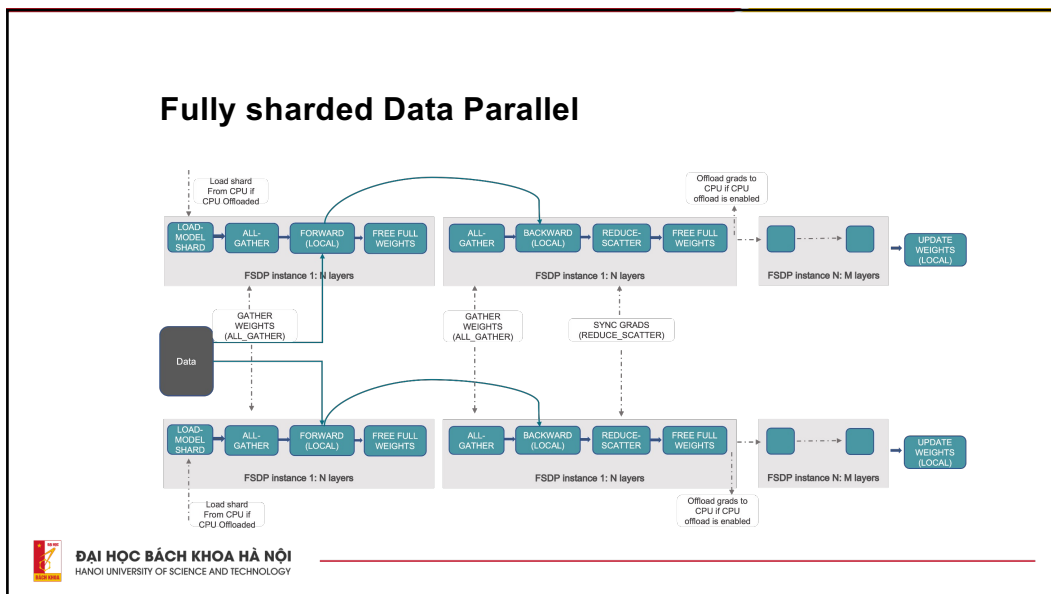


Data parallel





9



10

Comparison

Method	Pros	Cons
<u>Data parallel</u>	Simple to use	Slow due to replicas being destroyed
<u>Distributed data parallel</u>	Fast	High memory requirement
Fully sharded Data Parallel	Large models that other methods	Can be slower than DDP due to high communication cost



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

How to do it in Pytorch

💡 Datanparallel

```
- parallel_model = torch.nn.DataParallel(model)
```

💡 Distributed data parallel (DDP)

- Set a environment MASTER_ADDR and MASTER_PORT
- Initialize a process group
- parallel_model = nn.parallel.DistributedDataParallel(model, device_ids=[gpu])
- Use mp.spawn to spawn multiple processes
- ...

💡 Model parallelism

- Just don't



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

12

Instead use any high level framework

```

# run on cpu, gpu, tpu, ipu
# with no code changes needed
trainer = Trainer(devices=8, accelerator='cpu')
trainer = Trainer(devices=8, accelerator='gpu')
trainer = Trainer(devices=8, accelerator='tpu')
trainer = Trainer(devices=8, accelerator='ipu')

# or just let lightning auto detect
trainer = Trainer(devices=8, accelerator='auto')

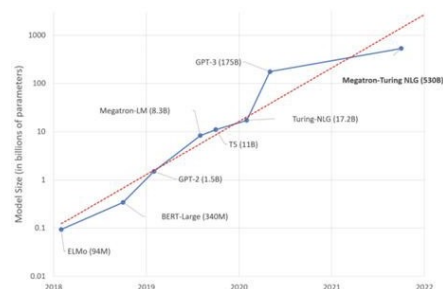
# for gpu, you can also do multiple nodes
# 32 nodes * 8 gpus per node = 256 gpus!
trainer = Trainer(devices=8, accelerator='gpu', num_nodes=32)

```

13

Above and beyond

Scaling matters in deep learning



```

# Sharded training using fairscale
trainer = Trainer(devices=4, strategy='ddp_sharded')

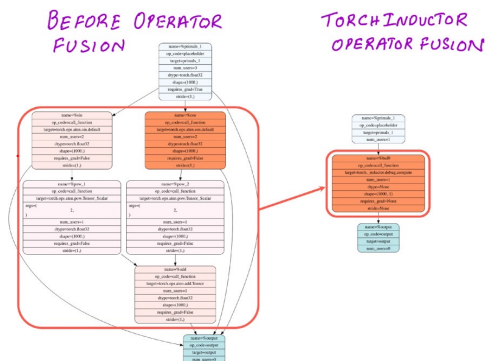
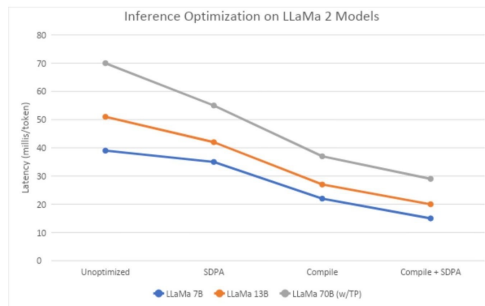
# sharded training using deepspeed
trainer = Trainer(devices=4, strategy="deepspeed_stage_1", precision=16)
trainer = Trainer(devices=4, strategy="deepspeed_stage_2", precision=16)
trainer = Trainer(devices=4, strategy="deepspeed_stage_3", precision=16)

```

14

Remember to compile your model

⚠ In Pytorch use `model = torch.compile(model)`



15

What about inference?

⚠ Use batch prediction when possible

```
from fastapi import FastAPI
from pipeline import model, clean_data, format_data, data_is_valid

app = FastAPI()

@app.post("/predict/")
async def predict(item):
    if not data_is_valid(item):
        return {"message": "data not valid"}

    item = clean_data(item)
    predictions = model.predict(item)
    output = format_data(predictions)

    return output
```

```
from fastapi import FastAPI
from typing import List
from pipeline import model, clean_data, format_data, data_is_valid

app = FastAPI()

@app.post("/batch-predict/")
async def predict(items: List[str]):
    items = list(set(items)) # <- remove duplicates

    items = [i for i in items if data_is_valid(i)] # <- leverage list comprehensions

    items = clean_data(items) # <- probably has some numpy or pandas
    predictions = model.predict(items) # <- faster and more efficient than call
    outputs = format_data(predictions)

    return outputs
```

16

What about inference?

⚠ Use caching if possible

```
import functools

@functools.lru_cache(maxsize=128)
def fib(n):
    if n < 2:
        return 1
    return fib(n-1) + fib(n-2)
```

```
$ python3 -m timeit -s 'from fib_test import fib' 'fib(30)'
10 loops, best of 3: 282 msec per loop
$ python3 -m timeit -s 'from fib_test import fib_cache' 'fib_cache(30)'
10000000 loops, best of 3: 0.0791 usec per loop
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

17

Exercise

This exercise is intended to be done on the [labeled faces in the wild \(LFW\)](#) dataset. The dataset consist images of famous people extracted from the internet. The dataset had been used to drive the field of facial verification, which you can read more about [here](#). We are going imagine that this dataset cannot fit in memory, and your job is therefore to construct a data pipeline that can be parallelized based on loading the raw datafiles (.jpg) at runtime.

- Download the dataset and extract to a folder. It does not matter if you choose the non-aligned or aligned version of the dataset.
- provide the `lfw_dataset.py` file where we have started the process of defining a data class. Fill out the `__init__`, `__len__` and `__getitem__`. Note that `__getitem__` expect that you return a single `img` which should be a `torch.Tensor`. Loading should be done using [PIL Image](#), as PIL images is the default input format for [torchvision](#) for transforms (for data augmentation).
- Make sure that the script runs without any additional arguments
`python lfw_dataset.py`
- Visualize a single batch by filling out the codeblock after the first *TODO* right after defining the dataloader. The visualization should show when launching the script as
`python lfw_dataset.py -visualize_batch`

Hint: this [tutorial](#).



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

18

Exercise

- Experiment how the number of workers influences the performance. We have already provide code that will pass over 100 batches from the dataset 5 times and calculate how long time it took, which you can play around with by calling

```
python lfw_dataset.py -get_timing -num_workers 1
```

Make a [errorbar plot](#) with number of workers along the x-axis and the timing along the y-axis. The errorbars should correspond to the standard deviation over the 5 runs. HINT: if it is taking too long to evaluate, measure the time over less batches (set the `-batches_to_check` flag). Also if you are not seeing an improvement, try increasing the batch size (since data loading is parallelized per batch).

For certain machines like the Mac with M1 chipset it is necessary to set the `multiprocessing_context` flag in the dataloader to "fork". This essentially tells the dataloader how the worker nodes should be created.

- Retry the experiment where you change the data augmentation to be more complex:

```
lfw_trans = transforms.Compose([
    transforms.RandomAffine(5, (0.1, 0.1), (0.5, 2.0)),
    # add more transforms here
    transforms.ToTensor()
])
```

by making the augmentation more computationally demanding, it should be easier to get an boost in performance when using multiple workers because the data augmentation is also executed in parallel.

